

CHAPTER 20

Logistic Regression

Logistic regression is a supervised machine learning algorithm developed for learning classification problems. A classification learning problem is when the target variable is categorical. The goal of logistic regression is to map a function from the features of the dataset to the targets to predict the probability that a new example belongs to one of the target classes. Figure 20-1 is an example of a dataset with categorical targets.

The diagram shows a table with four columns. The first three columns are grouped under the label 'input variables' with a bracket above them. The fourth column is labeled 'categorical target variable' with a downward arrow pointing to it. The table contains four rows of data. The first row has values 4, 6, 3, and 'glaucoma'. The second row has values 2, 4, 1, and 'cataract'. The third row has three dots in the first three columns and three dots in the fourth. The fourth row has values 5, 8, 5, and 'none'. To the right of the table, a bracket groups the rows and is labeled 'rows / records / entries'.

input variables			categorical target variable
x1	x2	x3	eye disease
4	6	3	glaucoma
2	4	1	cataract
...	...	...	...
5	8	5	none

Figure 20-1. Dataset with qualitative variables as output

Why Logistic Regression?

To develop our understanding of classification with logistic regression and why linear regression is unsuitable for learning categorical outputs, let us consider a binary or two-class classification problem. The dataset illustrated in Figure 20-2 has the output y (i.e., eye disease) = {disease, no-disease} is an example of dataset with binary targets.

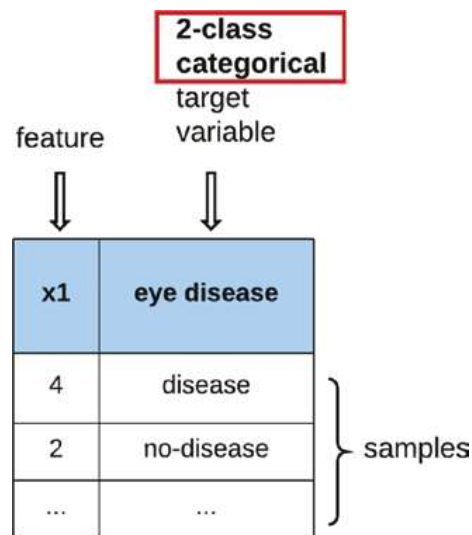


Figure 20-2. Two-class classification problem

From the illustration in Figure 20-3, the linear regression algorithm is susceptible to plot inaccurate decision boundaries especially in the presence of outliers (as seen toward the far right of the graph in Figure 20-3). Moreover, the linear regression model will be looking to learn a real-valued output, whereas a classification learning problem predicts the class membership of an observation using probability estimates.

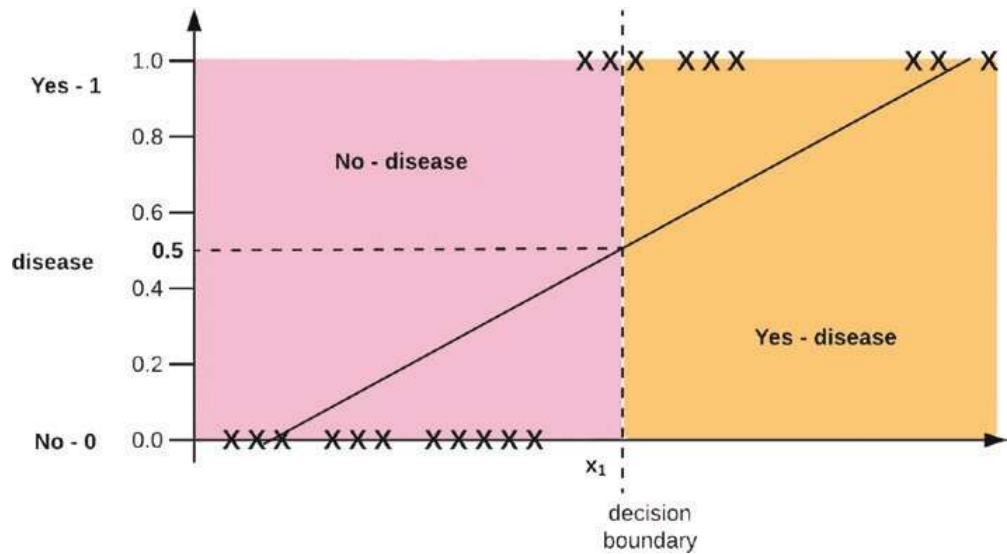


Figure 20-3. Linear regression on a classification dataset

Introducing the Logit or Sigmoid Model

The logistic function, also known as the logit or the sigmoid function, is responsible for constraining the output of the cost function so that it becomes a probability output between 0 and 1. The sigmoid function is formally written as

$$h(t) = \frac{1}{1 + e^{-t}}$$

The logistic regression model is formally similar to the linear regression model except that it is acted upon by the sigmoid model. The following is the formal representation:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$$h(\hat{y}) = \frac{1}{1 + e^{-\hat{y}}}$$

where $0 \leq h(t) \leq 1$. The sigmoid function is graphically shown in Figure 20-4.

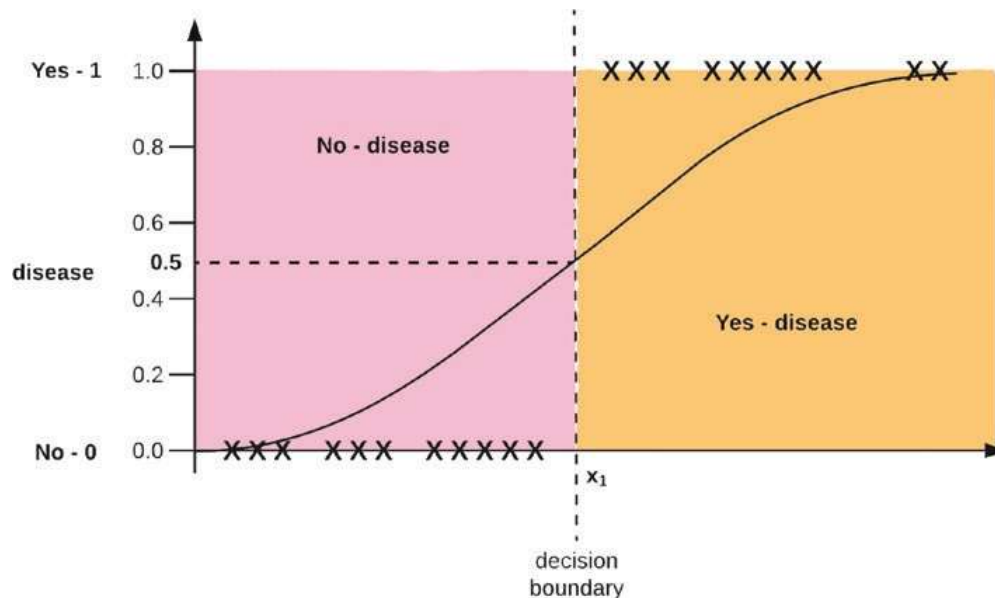


Figure 20-4. Logistic function

The sigmoid function, which looks like an S curve, rises from 0 and plateaus at 1. From the sigmoid function shown in Figure 20-4, as $\hat{y}$ increases to positive infinity, the sigmoid output gets closer to 1, and as t decreases toward negative infinity, the sigmoid function outputs 0.

Training the Logistic Regression Model

The logistic regression cost function is formally written as

$$\text{Cost}(h(t), y) = \begin{cases} -\log(h(t)) & \text{if } y = 1 \\ -\log(1 - h(t)) & \text{if } y = 0 \end{cases}$$

The cost function also known as **log-loss** is set up in this form to output the penalty of the algorithm if the model predicts a wrong class. To give more intuition, take, for example, a plot of $-\log(h(t))$ when $y = 1$ in Figure 20-5.

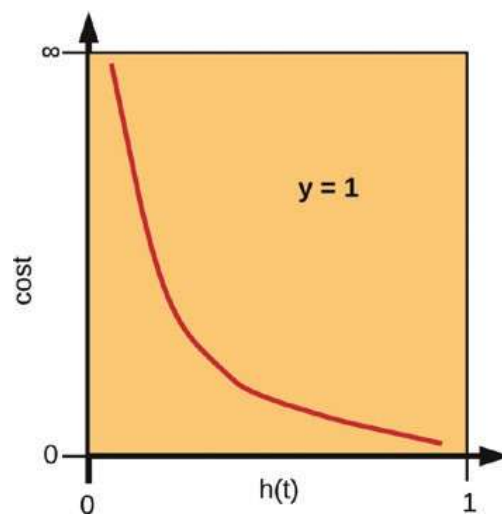


Figure 20-5. Plot of $h(t)$ when $y = 1$

In Figure 20-5, if the algorithm correctly predicts that the target is 1, then the cost tends toward 0. However, if the algorithm $h(t)$ predicts incorrectly the target as 0, then the cost on the model grows exponentially large. The converse is the case with the plot of $-\log(1 - h(t))$ when $y = 0$.

The logistic model is optimized using gradient descent to find the optimal values of the parameter θ that minimizes the cost function to predict the class with the highest probability estimate.

Multi-class Classification/Multinomial Logistic Regression

In multi-class or multinomial logistic regression, the labels of the dataset contain more than 2 classes. The multinomial logistic regression setup (i.e., the cost function and optimization procedure) is structurally similar to logistic regression; the only difference is that the output of logistic regression is 2 classes, while multinomial has greater than 2 classes (see Figure 20-6).

In Figure 20-6, the multi-class logistic regression builds a one-vs.-rest classifier to construct decision boundaries for the different class memberships.

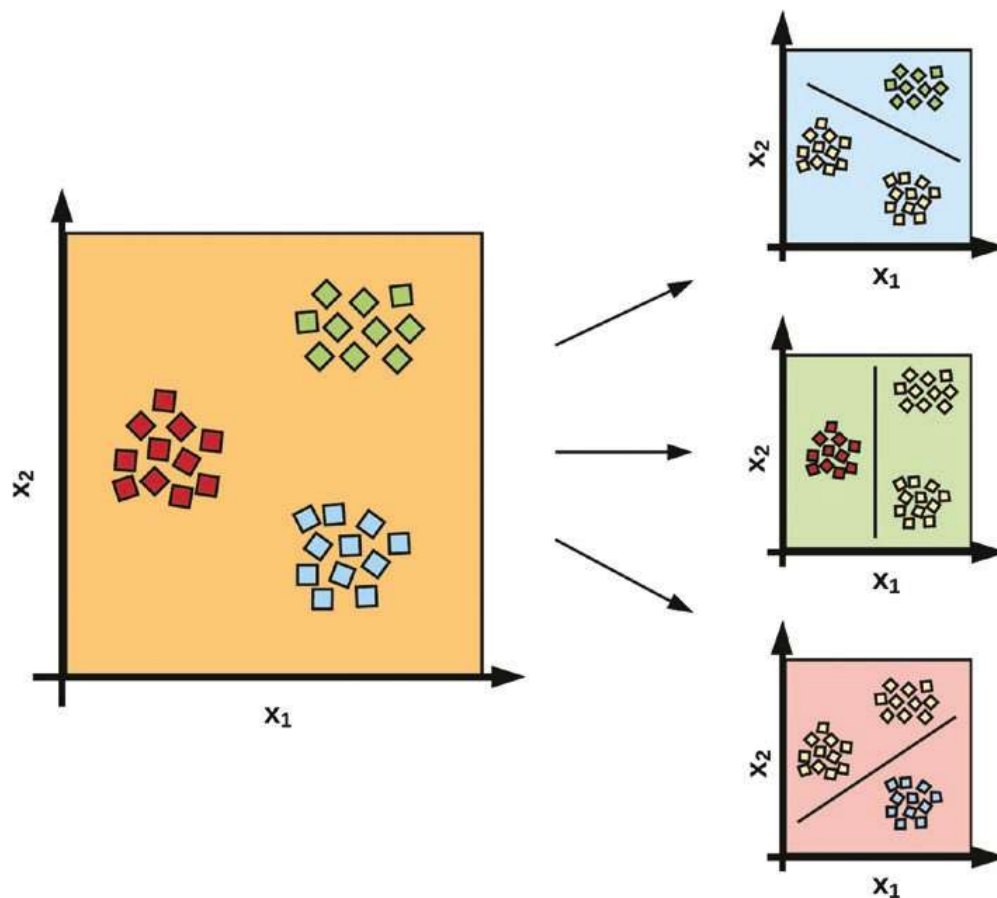


Figure 20-6. An illustration of multinomial regression

At this point, we introduce a critical function in machine learning called the softmax function. The softmax function is used to compute the probability that an instance belongs to one of the K classes when $K > 2$. We will see the softmax function show up again when we discuss (artificial) neural networks.

In order to build a classification model with k classes, the multinomial logistic model is formally defined as

$$\hat{y}(k) = \theta_0^k + \theta_1^k x_1 + \theta_2^k x_2 + \dots + \theta_n^k x_n$$

The preceding model takes into consideration the parameters for the k different classes. The softmax function is formally written as

$$p(k) = \sigma(\hat{y}(k))_i = \frac{e^{\hat{y}(k)_i}}{\sum_{j=1}^K e^{\hat{y}(k)_j}}$$

where

- $i = \{1, \dots, K\}$ classes.
- $\sigma(\hat{y}(k))_i$ outputs the probability estimates that an example in the training dataset belongs to one of the K classes.

The cost function for learning the class labels in a multinomial logistic regression model is called the ***cross-entropy*** cost function. Gradient descent is used to find the optimal values of the parameter θ that will minimize the cost function to ***predict the class with the highest probability estimate accurately***.

Logistic Regression with Scikit-learn

In this example, we will implement a multi-class logistic regression model with Scikit-learn. The model will predict the three species of flowers from the Iris dataset. The dataset contains 150 observations and 4 features. For this example, we use the accuracy metric and confusion matrix to access the model's performance.

```
# import packages
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import multilabel_confusion_matrix

# load dataset
data = datasets.load_iris()
# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
logistic_reg = LogisticRegression(solver='lbfgs', multi_class='ovr')

# fit the model on the training set
logistic_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = logistic_reg.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 0.97

# print the confusion matrix
multilabel_confusion_matrix(y_test, predictions)

'Output':
array([[26, 0],
       [ 0, 12]],

       [[25, 0],
       [ 1, 12]],

       [[24, 1],
       [ 0, 13]])

```

Take note of the following in the preceding code block:

- The logistic regression model is initialized by calling the method `LogisticRegression(solver='lbfgs', multi_class='ovr')`. The attribute `'multi_class'` is set to `'ovr'` to create a one-vs.-rest classifier.
- The confusion matrix for a multi-class learning problem uses the `'multilabel_confusion_matrix'` to calculate classwise confusion matrices where the labels are binned in a one-vs.-rest manner. As an example, the first matrix is interpreted as the difference between the actual and predicted targets for class 1 against other classes.

Optimizing the Logistic Regression Model

This section surveys a few techniques to consider in optimizing/improving the performance of logistic regression models.

In the case of Bias (i.e., when the accuracy is poor with training data)

- Remove highly correlated features. Logistic regression is susceptible to degraded performance when highly correlated features are present in the dataset.
- Logistic regression will benefit from standardizing the predictors by applying feature scaling.
- Good feature engineering to remove redundant features or recombine features based on intuition into the learning problem can improve the classification model.
- Applying log transforms to normalize the dataset can boost logistic regression classification accuracy.

In the case of variance (i.e., when the accuracy is good with training data, but poor on test data)

Applying regularization (more on this in [Chapter 21](#)) is a good technique to prevent overfitting.

This chapter provides a brief overview of logistic regression for building classification models. The chapter includes practical steps for implementing a logistic regression classifier with Scikit-learn. In the next chapter, we will examine the concept of applying regularization to linear models to mitigate the problem of overfitting.